Course-End Project: Python Adventure Game

Student Name: Sanjay Gosai

Tool: VS Code with GitHub Copilot

Language: Python 3.x

---

1. Project Overview

The "Ancient Treasure Quest" is a text-based adventure game where players navigate through different environments (a forest and a cave) to find a legendary treasure. The project focuses on utilizing Python functions, conditional logic, and iterative loops while leveraging GitHub Copilot for code optimization.

2. Technical Architecture

The game is structured using a **Modular Design**. Each location is isolated into its own function, which is then managed by a central game controller.

Game Logic (Decision Tree)

The player starts at a central hub and branches into different scenarios. Each scenario has a "Win," "Loss," or "Invalid Input" outcome.

3. GitHub Copilot Integration

GitHub Copilot acted as a pair programmer throughout the development. Its primary contributions included:

- **Logic Generation:** Copilot suggested the if-elif-else structures for handling player decisions.
- **Input Sanitization:** It is recommended using .lower().strip() to ensure that user inputs like " Forest" or "FOREST" would still be recognized by the program.
- **Error Handling:** Copilot assisted in creating a while loop for the main menu, ensuring that the game doesn't crash if a user enters an unrecognized command.

4. Key Programming Concepts Applied

The if __name__ == "__main__": Condition

A critical technical addition was the use of the __name__ == "__main__" block. This serves as a "Gatekeeper" for the script.

- **Purpose:** It ensures that the game loop only runs when the script is executed directly.
- **Benefit:** It allows the functions (like the forest or cave paths) to be imported into other scripts for testing without automatically starting the game intro.

The Restart Loop

To fulfil the requirement of allowing the player to restart, a while True loop was implemented in the main block. This allows the game to return to the beginning after a "Game Over" or "Win" screen unless the player explicitly chooses to exit.

5. Challenges and Enhancements

- **Challenge:** Managing player state across different functions.
- **Solution:** Passing the player_name variable as an argument to each function to maintain a personalised narrative.
- **Enhancement:** Added time.sleep() intervals to slow down the text delivery, creating a more immersive, "story-telling" atmosphere for the player.

6. Conclusion

Building this game with GitHub Copilot demonstrated how AI can accelerate development by handling boilerplate code and suggesting best practices like modularisation and input cleaning. The final script is robust, reusable, and easy to navigate.